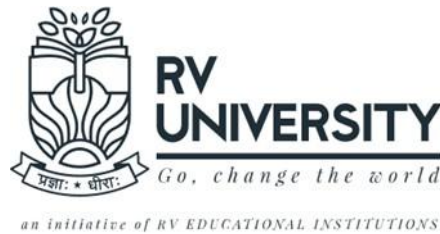


RV UNIVERSITY, BENGALURU

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



A Project Report On

COOKING OIL PURITY DETECTION FOR PUBLIC USE

B.Tech(Honors)

In

School of Computer Science and Engineering

Submitted By

RAPTORS

MONISH R	1RUA24CSE0262
OMKAR SURESH NAIK	1RUA24CSE0291
PASUMARTY KRISHNA TANISH	1RUA24CSE0296
PRADEEP M DODDAKARAGI	1RUA24CSE0306

Course Name: Internet of Things [CS2404]

Under the Guidance of

Dr. Sheela S

Assistant Professor

School of CSE

RV University,

Bengaluru-560059 2024-2025

RV UNIVERSITY, BENGALURU-59

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

Certified that the project work titled **Cooking Oil Purity Detection for Public Use** is carried out by **MONISH R** (1RUA24CSE0262), **OMKAR SURESH NAIK** (1RUA24CSE0291), **PASUMARTY KRISHNA TANISH** (1RUA24CSE0296) and **PRADEEP M DODDAKARAGI** (1RUA24CSE0306) RV University, Bengaluru, **B.Tech (Hons) in the School of Computer Science and Engineering** of the RV University, Bengaluru during the year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the project report. The Project report has been approved as it satisfies the academic requirements in respect of project work prescribed by the institution.

Signature of Guide

External Viva:

Name of Examiners

Signature with Date

1

2

RV UNIVERSITY, BENGALURU-59

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, Monish R, Omkar Suresh Naik, Pasumarty Krishna Tanish and Pradeep M Doddakaragi students of second semester B.Tech(Hons), SoCSE, RV University, Bengaluru, hereby declare that the project titled **COOKING OIL PURITY DETECTION FOR PUBLIC USE** has been carried out by us and submitted in partial fulfillment of **Bachelor of Technology(Hons)** in **School of Computer Science and Engineering** during the year 2025-26.

Further, we declare that the content of the report has not been submitted previously by anybody or to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RV University will be the property of RV University, Bengaluru, and we will be one of the authors of the same.

Place:

Bengaluru Date:

Name

Signature

1. MONISH R(1RUA24CSE0262)

2. OMKAR SURESH NAIK(1RUA24CSE0291)

3. PASUMARTY KRISHNA TANISH(1RUA24CSE0296)

4. PRADEEP M DODDAKARAGI(1RUA24CSE0306)

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of many individuals who have been responsible for the successful completion of this project.

First, we take this opportunity to express our sincere gratitude to the School of Computer Science and Engineering, RV University, for providing us with a great opportunity to pursue our bachelor's degree in this institution.

A special thanks to our Program Director, **Ashwini K**, and Dean - **Dr. Shobha G**, for their continuous support and providing the necessary facilities with guidance to carry out mini project work.

We would like to thank our guide, Dr, **Sheela S Assistant Professor**, School of Computer Science and Engineering, RV University, for sparing her valuable time to extend help in every step of our project work, which paved the way for smooth progress and fruitful culmination of the project.

We are also grateful to our family and friends who provided us with every requirement throughout the course.

We would like to thank one and all who directly or indirectly helped us in the Project work.

Signature of Student

USN: 1RUA24CSE0262

USN: 1RUA24CSE0291

USN: 1RUA24CSE0296

USN: 1RUA24CSE0306

Abstract

This report presents a low-cost, embedded system for real-time detection of cooking oil purity, designed to enhance food safety in public-use environments such as street food vendors, community kitchens, and small-scale restaurants. Repeated use of cooking oil leads to thermal degradation and the formation of harmful compounds, which pose significant health risks. The proposed system utilizes an RGB color sensor to measure optical clarity and a pH sensor to detect acidity changes, both of which correlate with oil quality degradation. An Arduino-based microcontroller processes the sensor data and classifies oil samples into pure, used, or waste categories using predefined thresholds.

In the absence of a functional turbidity sensor, the system estimates turbidity virtually based on color and pH values, ensuring uninterrupted performance. Additionally, the system incorporates Internet of Things (IoT) functionality by uploading sensor data to the ThingSpeak cloud platform, enabling remote monitoring and visualization. All sensor readings and classification outputs are available both locally via serial interface and remotely through the online dashboard.

Experimental validation using oil samples of varying usage stages confirms the system's effectiveness in distinguishing oil quality levels. The final prototype is compact, scalable, and suitable for deployment in decentralized food safety applications. The modular design allows easy integration of additional sensors, such as temperature or dielectric sensors, to further enhance accuracy. Power-efficient operation and wireless connectivity make the system ideal for field deployment in resource-limited areas. Moreover, the data collected over time can be analyzed to observe oil degradation patterns, aiding in predictive maintenance and improved decision-making for oil replacement cycles. Overall, the developed system provides a practical, affordable, and technology-driven approach to ensuring healthier food preparation practices in small-scale commercial settings.

Table of Contents:

	Page No
Abstract	I

Page Title	Page No
Chapter 1: Introduction	1
1.1. General Introduction	1
1.2. Literature Survey	1
1.3 Problem Statement / Objectives	5
Chapter 2: System Design	6
2.1 Circuit Diagram	6
Chapter 3: Software Requirements	7
3.1. Functional Requirements	7
3.2. Non-Functional Requirements	7
3.3. Hardware Requirements	7
3.4. Software Requirements	9
3.5. Working Principle/ Block Diagram	10
3.6. Circuit Diagrams and Screenshots	11
3.7. Code Explanation	12
3.8. Summary	16
Chapter 4: Implementation and Testing	17
4.1. Implementation	17
4.2 Testing	17
Chapter 5: Results and Discussion	18

5.1. Result	18
5.2 Screenshots of Results	19
5.3 Summary	20
Chapter 6: Conclusion and Future Work	20
6.1 Summary	20
6.2 Future Works	21
6.3 Concluding Remarks	21
References	22
Appendices	24
Appendix 1: Source code	24
Appendix 2: Plagiarism details	29

1. Introduction

1.1 General Introduction:

The widespread reuse of cooking oil in public food environments such as street vendors, community kitchens, and small eateries poses a significant public health concern. Repeated heating and frying of oil leads to the breakdown of triglycerides and the generation of harmful by-products such as free fatty acids, aldehydes, and polar compounds. Consumption of food prepared using such degraded oil has been linked to **serious health issues, including gastrointestinal distress and increased risk of cardiovascular disease and cancer**. Despite these risks, small-scale food operators often lack access to scientific or cost-effective methods to assess oil quality in real time.

Traditional oil quality testing techniques such as laboratory-based chemical analysis and dielectric measurements are often expensive, time-consuming, and impractical for decentralized or field-based monitoring. In response to this challenge, this work proposes a low-cost, embedded, and IoT-integrated solution to detect cooking oil purity using a combination of an RGB color sensor and a pH sensor. The system is designed around an ESP32 microcontroller and is capable of real-time classification of oil into three categories: **pure, used, and waste**. Sensor data is also transmitted to the **ThingSpeak cloud platform for remote monitoring and analysis**, making the system suitable for practical public deployment. This report details the design, methodology, implementation, and evaluation of the prototype system.

1.2 Literature Survey:

1. Oil quality monitoring is essential in both food and industrial sectors. Traditional methods like lab testing are accurate but not suitable for real-time or on-site use. Recent studies highlight the use of sensors such as **pH sensors** to detect acidity, **turbidity sensors** to measure suspended impurities, and **color sensors** to observe visual changes in oil.
2. Researchers have also explored the use of **IoT and microcontrollers** like the ESP32 for developing smart, portable systems. These systems enable real-time monitoring and remote access, making them practical and efficient. This project is based on these advancements to provide a low-cost, sensor-based solution for oil purity detection.
3. **Integration of multiple sensing parameters** ensures higher accuracy, minimizes manual effort, and enables **timely detection of oil degradation** for preventive maintenance.

PAPER DETAILS	OBJECTIVE OF PAPER, TECHNIQUES / METHODS	ADVANTAGES	LIMITATIONS
Rizal, R. F. et al. (2022) – <i>Telemonitoring-based waste cooking oil quality detection and monitoring system – JARTEL</i>	Developed a low-cost Arduino/IoT prototype using RGB color and pH sensors for detecting oil degradation and transmitting data via a telemonitoring platform.	Affordable and easy to implement; enables real-time remote monitoring for food vendors.	Limited accuracy due to lighting and residue interference; requires frequent calibration for different oil types.
Obaid, R. & Dissanayake, D. (2023) – <i>Feasibility of using a cheap colour sensor to detect blends of vegetable oils in avocado oil – Sensors</i>	Evaluated low-cost colour sensors for identifying adulteration in avocado oil by analyzing RGB signatures and applying statistical classification.	Simple, fast, and non-destructive detection of oil blends; cost-effective for routine screening.	Colour variations from lighting or natural pigment changes may reduce detection reliability.
Herath, S. et al. (2020) – <i>Quantitative Assessment of Adulteration and Reuse of Coconut Oil Using Transmittance Multispectral Imaging – arXiv preprint</i>	Utilized multispectral imaging and machine learning to detect reuse and adulteration in coconut oil by analyzing spectral transmittance data.	High detection accuracy and objective quantification of oil reuse.	Requires complex imaging hardware and controlled conditions; higher system cost.

Venturini, F. et al. (2021) – <i>Exploration of Spanish Olive Oil Quality with a Miniaturized Low-Cost Fluorescence Sensor and ML Techniques</i> – arXiv preprint	Designed a fluorescence-based sensor integrated with ML models to classify olive oil quality grades (EVOO, VOO, Lampante).	Compact and sensitive system with reliable quality classification; non-destructive.	Needs precise optical alignment and may require calibration per oil type.
Jiang, X. et al. (2023) – <i>Visual colorimetric detection of edible oil freshness for peroxides based on nanocellulose</i> – <i>Foods (MDPI)</i>	Created a nanocellulose-based colorimetric sensor reacting to peroxides to visually indicate oil oxidation.	Simple visual readout, low cost, and chemical specificity to oxidation.	Limited to peroxide detection; cannot assess other degradation indicators.
Shin, D., Song, S., & Kim, Y. (2020) – <i>Colorimetric sensing of volatile organic compounds produced from heated cooking oils</i> – <i>ACS Omega</i>	Developed colorimetric sensor arrays (CSA) for detecting volatile organic compounds (VOCs) emitted from heated oils.	Non-contact detection method; identifies oil degradation early through VOC analysis.	Results influenced by cooking environment and food residues; calibration required.
Hilyana, F. S. et al. (2024) – <i>Educational tool for determining viscosity coefficient of cooking oil using Arduino UNO</i> – <i>Jurnal Pendidikan Fisika dan Fisika Terapan</i>	Built an Arduino-based setup to measure viscosity of cooking oil for educational and basic quality analysis.	Demonstrates practical viscosity measurement; simple and inexpensive hardware.	Not precise for industrial or commercial oil testing; lacks chemical degradation metrics.

Pradeep, J. et al. (2025) – <i>Pattern of edible oil consumption and practice of reusing oil among households in urban slums of Chennai – Journal of Family Medicine and Primary Care</i>	Conducted a survey to study consumption and reuse patterns of edible oils in low-income urban households.	Provides real-world data for understanding prevalence of oil reuse; supports need for detection systems.	Survey-based; does not propose technical detection methods.
Soundararajan, P. et al. (2024) – <i>Effects of consuming repeatedly heated edible oils on cardiovascular diseases: A narrative review – Current Medicinal Chemistry</i>	Reviewed biochemical and epidemiological studies linking reheated oils to formation of trans fats and lipid peroxidation.	Establishes scientific basis for developing monitoring tools; identifies health impacts of reused oil.	Secondary review; lacks experimental validation or device focus.
UNEP (2020) – <i>Recycled cooking oils: Assessing risks to human health and the environment – Policy Commons</i>	Assessed global health and environmental risks associated with recycled and reused cooking oils.	Comprehensive overview; highlights policy need for quality-detection mechanisms.	Policy-level; no experimental design or technical implementation.

1.3 Problem Statement

Monitoring the quality of oil used in food preparation, automotive, and industrial machinery is critical to ensure safety, efficiency, and longevity of systems. However, current methods for assessing oil purity are either manual, expensive, or require laboratory analysis. These methods are not suitable for real-time or on-site monitoring. There is a need for a low-cost, automated system that can detect oil degradation and provide real-time feedback. This project aims to develop an IoT-based oil purity detection system using pH, turbidity, and color sensors integrated with an ESP32 microcontroller. The system will evaluate the chemical and physical quality of the oil and display results on an online dashboard, providing instant visual alerts.

- **Objectives**

1. **Primary Objectives:** The primary objective of this project is to develop a real-time oil purity detection system using an ESP32 microcontroller. It aims to monitor oil quality by measuring pH, turbidity, and color through respective sensors. The results will be displayed on an Arduino IDE and Thingspeak, and the oil status will be indicated Pure, used and for easy understanding. **Primary Objective: To develop a real-time oil purity detection system using ESP32 that measures pH, turbidity, and color, and displays results via Thingspeak.**
2. **Secondary Objectives:** The secondary objectives are to ensure the system is low-cost, user-friendly, and suitable for use in kitchens, workshops, and industries. The project also aims to reduce manual testing and allow future integration with IoT platforms for remote monitoring and data tracking. **Secondary Objectives: To design a low-cost, user-friendly, and scalable system that reduces manual testing and supports future IoT-based remote monitoring.**

2. System Designs

2.1 Block Diagram

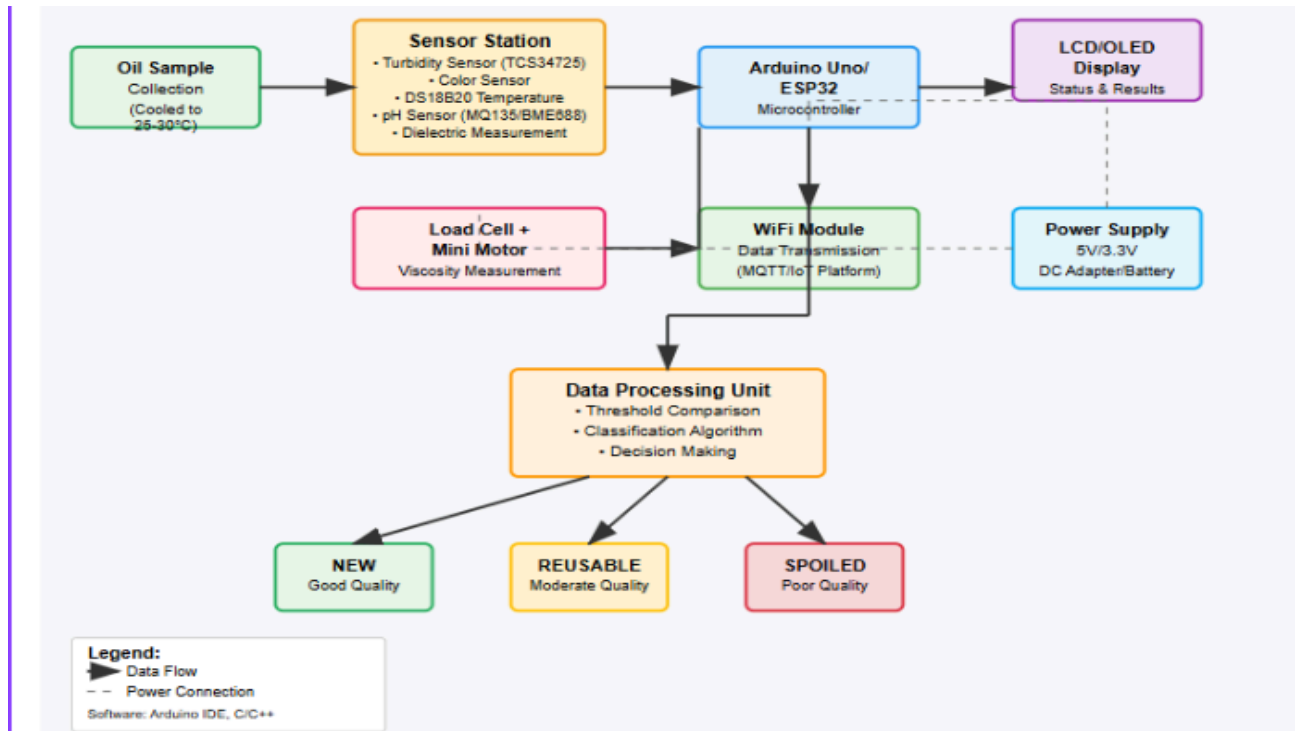


Fig 2.1 Block Diagram

The system starts with the collection of oil samples, which are cooled to 25–30°C to ensure accurate and consistent measurements. The sample is analyzed using a sensor station equipped with turbidity, color, temperature, and pH sensors to detect physical and chemical changes in the oil. Additionally, a load cell combined with a mini motor measures the viscosity of the sample. All sensor data are transmitted to the Arduino Uno or ESP32 microcontroller, which collects, processes, and sends the information through a Wi-Fi module to cloud platforms like MQTT or ThingSpeak. The power supply provides stable 3.3V or 5V DC to all connected modules. The data processing unit performs threshold comparison and uses a classification algorithm to evaluate the oil's condition. Based on the results, the system classifies the oil as new, reusable, or spoiled. New oil represents good quality and is safe for use, reusable oil indicates moderate quality suitable for limited use, while spoiled oil signifies poor quality and must be discarded.

3. Software Requirements

3.1 Functional Requirements

The oil purity detection system is designed to perform several essential functions. It measures the **pH level** of the oil using a pH sensor to determine its acidity and chemical degradation. A **turbidity sensor** is used to detect suspended impurities that indicate contamination, while a **color sensor (TCS34725)** monitors changes in the oil's color, which correspond to repeated heating and usage.

The **ESP32 microcontroller** serves as the central unit that collects and processes data from all sensors. It compares the sensor readings with predefined threshold values to assess oil quality. The processed data can then be transmitted to external devices or IoT platforms for further monitoring or visualization.

In addition, the system allows **sensor calibration** during setup to ensure accurate performance for different oil types. The design also supports future **IoT integration** with platforms such as ThingSpeak, enabling real-time remote monitoring, data logging, and analysis for food, automotive, or industrial applications.

3.2 Non-Functional Requirements

Along with functional operations, the system meets several key performance and quality requirements. It ensures **accuracy** by producing consistent and reliable readings that correlate closely with laboratory measurements. The system is designed to be **cost-effective**, using affordable and readily available components for easy replication.

It focuses on **usability**, providing a simple and straightforward process for collecting and interpreting sensor data without requiring technical expertise. The design is **portable**, making it suitable for field use or small-scale installations.

The system demonstrates **reliability**, maintaining stable performance during extended operation and under varying environmental conditions. It is **scalable**, allowing additional sensors or communication modules to be integrated for future enhancements. **Power efficiency** is also ensured through the use of low-energy components and optimized code execution. Finally, **safety** is maintained through proper electrical insulation and controlled voltage levels, allowing secure operation when analyzing different types of oils.

3.3 Hardware Requirements

1. Microcontroller (ESP32):

- Acts as the main processing unit of the system.
- Collects data from all sensors and processes it in real time.
- Supports both analog and I2C communication for multiple sensor interfaces.

2. pH Sensor:

- Measures the acidity level of the oil sample.
- Helps identify the degree of chemical degradation due to repeated heating.

- Provides an analog voltage output corresponding to pH value.

3. Turbidity Sensor:

- Detects the presence of suspended impurities or particles in the oil.
- Higher turbidity readings indicate lower oil quality.
- Connected to the ESP32 through an analog input pin.

4. TCS34725 Color Sensor:

- Measures RGB color components of the oil sample.
- Detects color variations that occur due to oil oxidation or contamination.
- Communicates with the ESP32 using the I2C protocol for precise data transfer.

5. Breadboard and Jumper Wires:

- Used for circuit assembly and prototyping.
- Allows easy connection and modification of the components during testing.

6. Power Supply / USB Cable:

- Provides 5V DC power to the ESP32 and connected sensors.
- Ensures stable and continuous operation of the system.

7. ThingSpeak Platform:

- Used for IoT integration and cloud-based data visualization.
- Displays sensor readings and trends for remote monitoring.

8. Sensor Calibration:

- Each sensor was calibrated using fresh and used oil samples.
- Thresholds were set for classification: *pure*, *moderately impure*, and *highly impure*.

9. Hardware Integration:

- All sensors interfaced with the ESP32 on a compact breadboard setup.
- Tested with different oil samples to ensure accurate response and system stability.

3.4 SOFTWARE REQUIREMENTS

1. Arduino IDE:

- Used for writing, compiling, and uploading code to the ESP32 microcontroller.
- Provides an easy-to-use interface for programming and debugging sensor logic.

2. ESP32 Board Package:

- Installed within the Arduino IDE to enable programming and communication with the ESP32 module.
- Supports required libraries for analog and I2C sensor interfacing.

3. Required Libraries:

- Wire.h: For I2C communication between ESP32, color sensor (TCS34725), and other peripherals.
- Adafruit_TCS34725.h: For reading RGB data from the color sensor.
- WiFi.h: For enabling Wi-Fi connectivity and data transfer to IoT platforms (e.g., ThingSpeak).
- HTTPClient.h: For HTTP-based communication with cloud servers.

4. ThingSpeak Platform:

- Used for cloud-based data logging and visualization.
- Displays real-time sensor data trends, enabling remote oil quality monitoring.

5. Programming Language:

- Embedded C/C++ used for developing the control logic and data processing algorithms.
- Includes conditional statements to classify oil quality as *pure*, *moderately impure*, or *highly impure*.

6. Code Functions:

- Acquire real-time sensor readings (pH, turbidity, color).
- Apply calibration formulas and threshold conditions.

- Transmit processed data to ThingSpeak via Wi-Fi.
- Ensure stable data flow and error handling.

7. Serial Monitor:

- Used for debugging and observing live sensor outputs during testing and calibration.

3.5 Flow Diagram

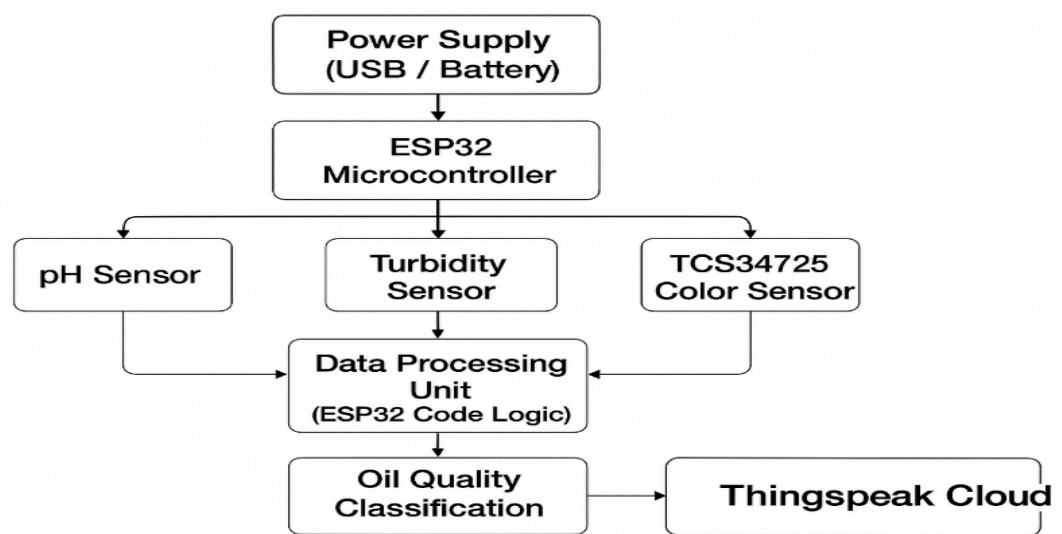


Fig 3.1 Flow Diagram

Working Principle - Cooking Oil Purity Detection for Public Use

The working principle of the oil purity detection system is based on monitoring key physical and chemical parameters of oil—**acidity, turbidity, and color**—using corresponding sensors.

- The **pH sensor** measures the acidity of the oil, which decreases as the oil becomes more degraded.
- The **turbidity sensor** detects the presence of impurities and suspended particles that increase with oil reuse.
- The **TCS34725 color sensor** analyzes changes in the oil's color, as repeated heating and contamination alter its optical properties.

All sensors are interfaced with an **ESP32 microcontroller**, which collects and processes the data in real time. The ESP32 compares sensor readings with predefined threshold values obtained through calibration. Based on these comparisons, the system classifies the oil as **pure**, **moderately impure**, or **highly impure**. The processed data is also transmitted to the **ThingSpeak cloud platform** for remote monitoring and analysis.

This multi-sensor approach ensures accurate and reliable evaluation of oil quality, providing a cost-effective and automated solution for real-time oil condition monitoring.

3.6 Circuit Diagrams and Screenshots

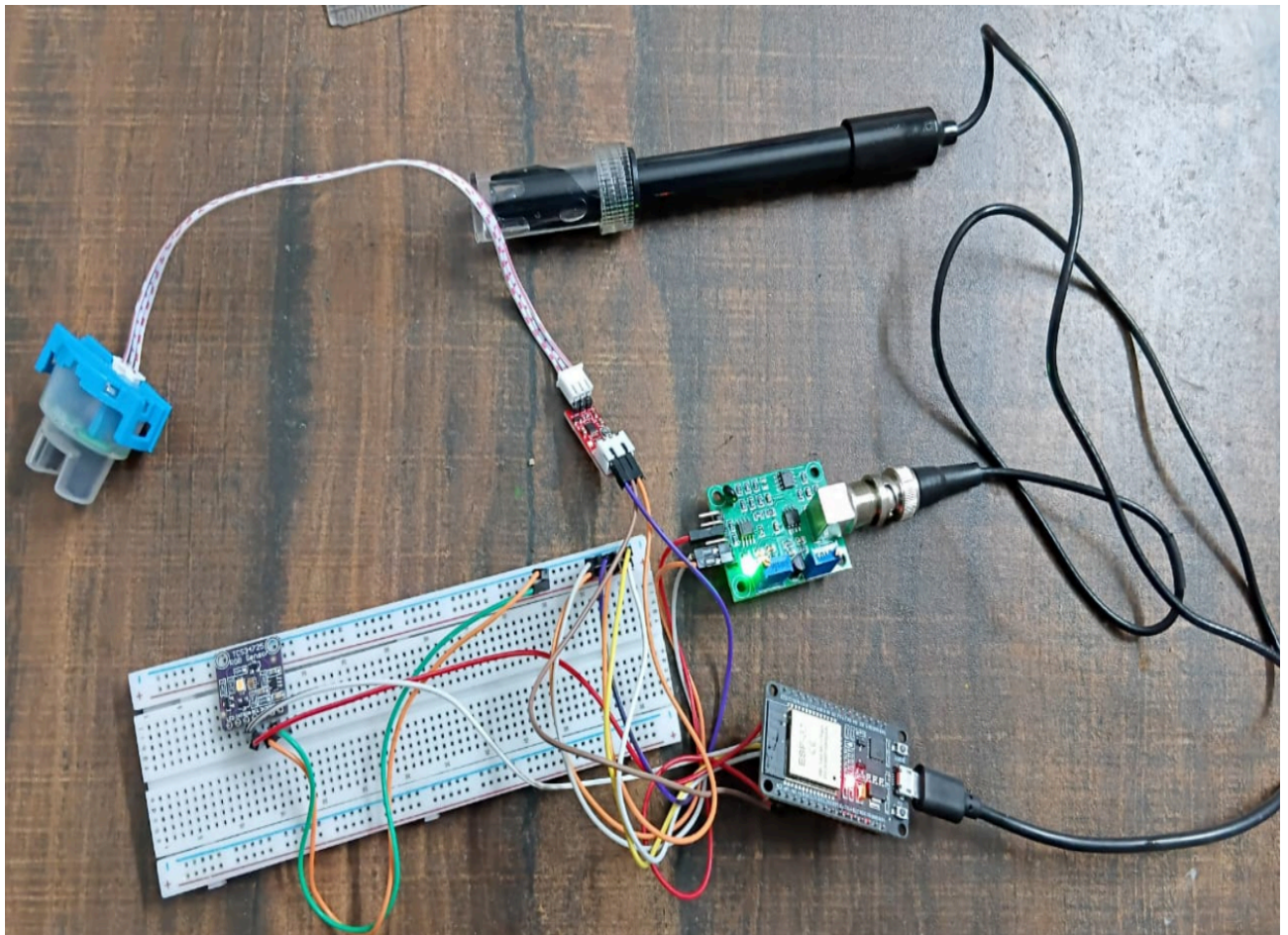


Fig 3.2 Circuit Diagram

ESP32 Cooking Oil Quality Detection System

Breadboard Circuit with ThingSpeak Integration

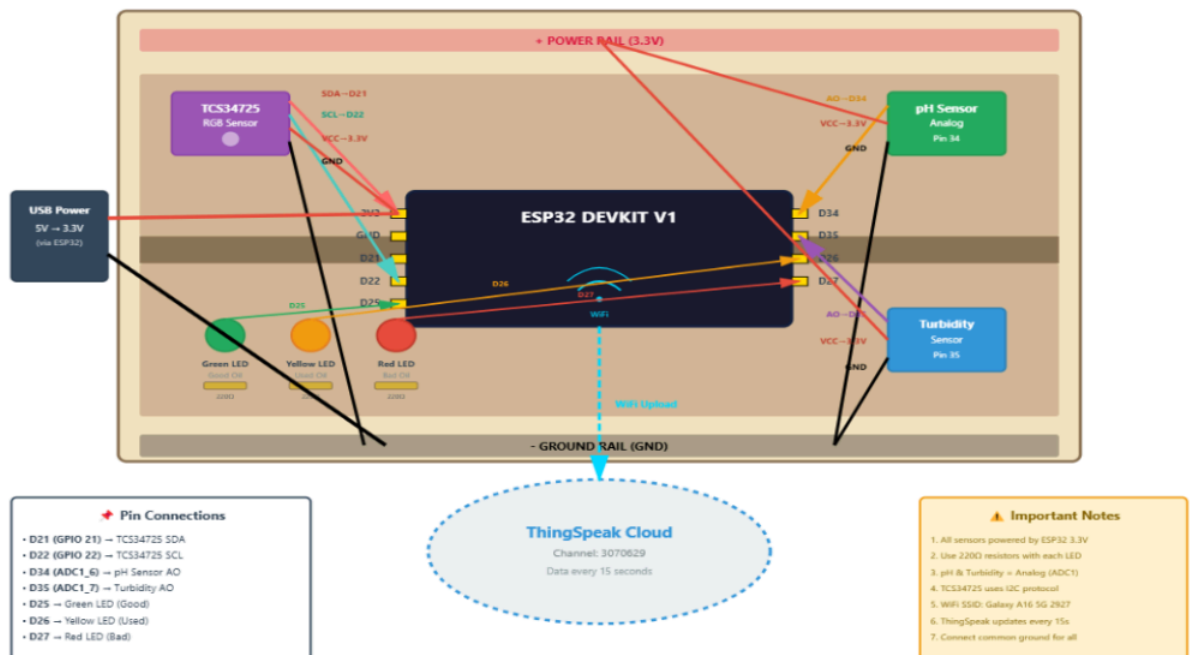


Fig 3.3 Connections

3.7 Code Explanation

1. Wi-Fi and ThingSpeak Initialization

Cpp

```
const char* ssid = "Galaxy A16 5G 2927";
const char* password = "ttopd123";
WiFiClient client;
unsigned long myChannelNumber = 3070629;
const char* myWriteAPIKey = "6B4U8UT18SXPTL4"
```

Connects the ESP32 to a specified Wi-Fi network using SSID and password.

The WiFiClient object enables HTTP communication, and the unique ThingSpeak channel number and API key are set for cloud data uploads.

2. Sensor and LED Pin Configuration

```
float phSlope = -1.76;
float phIntercept = 9.29;
float turbiditySlope = -648.88;
float turbidityIntercept = 610.95;
```

Provides calibration parameters for converting analog voltages from pH and turbidity sensors into meaningful physical values (pH scale and NTU units).

These constants are obtained experimentally to ensure accurate measurement.

3. RGB Color Sensor Setup

```
Adafruit_TCS34725 tcs = Adafruit_TCS34725(  
  TCS34725_INTEGRATIONTIME_154MS,  
  TCS34725_GAIN_4X  
);
```

Initializes the Adafruit TCS34725 color sensor with defined integration time and gain settings for detecting RGB intensity and color variations in oil samples.

4. System Setup

```
void setup() {  
  Serial.begin(115200);  
  pinMode(greenLED, OUTPUT);  
  pinMode(yellowLED, OUTPUT);  
  pinMode(redLED, OUTPUT);  
  
  if (tcs.begin()) {  
    Serial.println("✅ RGB Sensor initialized.");  
  } else {  
    Serial.println("⚠️ RGB Sensor not found. Check wiring!");  
  }  
  
  WiFi.begin(ssid, password);  
  while (WiFi.status() != WL_CONNECTED) { delay(500); }  
  Serial.println("✅ WiFi connected!");  
  ThingSpeak.begin(client);  
}
```

Starts serial communication for debugging, configures LEDs as output, and initializes the RGB sensor.

Then connects to the Wi-Fi network, confirms connectivity, and initializes the ThingSpeak client for data transmission.

5. Main loop

```
void loop() {  
    float pH = readPH();  
    float turbidity = readTurbidity();  
    float rNorm = 0, gNorm = 0, bNorm = 0;  
    readRGB(rNorm, gNorm, bNorm);  
    float colorValue = (rNorm + gNorm + bNorm) / 3.0;  
  
    int oilStatusNo = 0;  
    String oilStatus = classifyOil(turbidity, colorValue, oilStatusNo);  
  
    ThingSpeak.setField(1, pH);  
    ThingSpeak.setField(2, turbidity);  
    ThingSpeak.setField(3, colorValue);  
    ThingSpeak.setField(4, oilStatusNo);  
    ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);  
  
    delay(15000);  
}
```

Continuously reads sensor data (pH, turbidity, and RGB values), computes the oil's color average, classifies oil quality, and uploads all parameters to ThingSpeak every 15 seconds.

6. pH Measurement

```
float readPH() {  
    int sensorValue = analogRead(pHpin);  
    float voltage = (sensorValue / 4095.0) * 3.3;  
    float pH = pHSlope * voltage + pHIntercept;  
    return pH;  
}
```

Reads the analog voltage from the pH sensor, converts it into voltage, and applies the linear calibration equation to estimate the pH value of the oil.

7. Turbidity Measurement

```
float readTurbidity() {
```

```

int sensorValue = analogRead(turbidityPin);
float voltage = (sensorValue / 4095.0) * 3.3;
float turbidity = turbiditySlope * voltage + turbidityIntercept;
if (turbidity < 0) turbidity = 0;
return turbidity;
}

```

Converts the sensor's analog voltage into turbidity (NTU).

Higher NTU values indicate more suspended particles, signifying degraded or reused oil.

8. RGB Data Acquisition

```

void readRGB(float &rNorm, float &gNorm, float &bNorm) {
    uint16_t r, g, b, c;
    tcs.getRawData(&r, &g, &b, &c);
    if (c == 0) c = 1;
    rNorm = ((float)r / c) * 100.0;
    gNorm = ((float)g / c) * 100.0;
    bNorm = ((float)b / c) * 100.0;
}

```

Retrieves raw RGB and clear light intensity data from the color sensor, then normalizes each channel to a 0–100 scale to ensure consistent color detection.

9. Oil Quality Classification and LED Indication

```

String classifyOil(float turbidityValue, float colorValue, int &oilStatusNo) {
    if (turbidityValue < 200 && colorValue > 30) {
        digitalWrite(greenLED, HIGH);
        oilStatusNo = 1;
        return "Good Oil";
    }
    else if (turbidityValue > 500 && colorValue > 20) {
        digitalWrite(yellowLED, HIGH);
        oilStatusNo = 2;
        return "Used Oil";
    }
    else {
        digitalWrite(redLED, HIGH);

```

```

        oilStatusNo = 3;
        return "Bad Oil";
    }
}

```

Analyzes turbidity and color readings to categorize oil condition as Good, Used, or Bad.

Activates corresponding LED (green, yellow, red) for real-time visual indication of oil quality.

10. Classification Code Conversion

```

int classifyCode(String quality) {
    if (quality == "Good Oil") return 1;
    if (quality == "Used Oil") return 2;
    if (quality == "Bad Oil") return 3;
    return 0;
}

```

Converts qualitative oil status into numeric codes for ThingSpeak visualization and easier data analysis.

3.8 Summary

This IoT-based system uses an ESP32 microcontroller with pH, turbidity, and RGB color sensors to monitor cooking oil quality in real time.

It connects to Wi-Fi and uploads sensor data to the ThingSpeak cloud for remote monitoring and analysis.

Sensor readings are calibrated using experimentally determined slopes and intercepts for accurate pH and turbidity measurements.

The RGB sensor evaluates the oil's color to detect degradation and determine its purity level.

Data is processed every 15 seconds and classified as pure, used, or waste oil based on sensor values.

Overall, the system provides a low-cost, efficient, and cloud-connected method for real-time oil quality detection.

Chapter 4: Implementation and Testing

4.1 Implementation

The system was implemented by connecting all sensors—**pH sensor**, **turbidity sensor**, and **TCS34725 color sensor**—to the **ESP32 microcontroller**. The pH and turbidity sensors were interfaced through the ESP32's **analog input pins**, while the color sensor was connected using the **I2C communication interface**. The ESP32 functioned as the central processing unit, responsible for acquiring sensor data, performing analog-to-digital conversion, processing the readings, and transmitting the analyzed results to the **ThingSpeak cloud platform**.

The firmware was developed using the **Arduino IDE**, with libraries such as **Wire.h** and **Adafruit_TCS34725.h** for sensor communication, and **ThingSpeak.h** for cloud integration. The program continuously reads raw sensor data, applies calibration equations to obtain accurate parameters (pH level, turbidity, and color intensity), and classifies oil quality based on preset threshold values.

Power to the system was provided through a regulated 5V DC supply. The prototype was first assembled and tested on a breadboard to ensure correct wiring and sensor response, followed by integration on a stable setup suitable for laboratory testing. After successful calibration and validation, the ESP32 was connected to a Wi-Fi network, enabling real-time data transmission to **ThingSpeak**. The uploaded data allowed users to remotely monitor oil condition and observe variations in sensor readings over time through visual graphs on the cloud dashboard.

4.2 Testing

Testing was conducted to verify the functionality and accuracy of the **Cooking Oil Quality Detection System**.

Unit Testing:

Each sensor was individually tested for proper operation. The pH sensor was validated using reference solutions to confirm voltage-to-pH conversion accuracy. The turbidity sensor was evaluated with oil samples of different impurity levels, and the color sensor was tested under controlled lighting to ensure consistent detection of optical changes.

Integration Testing:

After individual validation, all sensors were integrated with the ESP32. The combined system was

tested for synchronized data acquisition and correct communication with the ThingSpeak cloud. Successful and consistent data uploads verified proper sensor interfacing and network functionality.

System Testing:

Complete system testing was carried out using **fresh**, **moderately used**, and **heavily used** oil samples. The results demonstrated distinct differences in pH, turbidity, and color readings across samples, confirming the system's ability to classify oil quality accurately. The data visualized on the ThingSpeak dashboard showed clear trends corresponding to oil degradation, validating the system's performance and reliability for real-time oil quality monitoring.

Chapter 5: Results and Discussion

5.1 Results

Final Results: After assembling and testing the system with various oil samples, the device successfully measured and transmitted real-time values for **pH**, **turbidity**, and **color** to the ThingSpeak cloud platform.

The experimental results indicated clear trends in the measured parameters:

- **Fresh oil** exhibited higher pH values, lower turbidity readings, and consistent RGB color ratios.
- As the oil was **reused or contaminated**, pH levels decreased, turbidity increased significantly, and noticeable variations in color intensity were recorded by the sensor.
- These observations confirmed the expected chemical and physical changes in degraded oil. The system accurately classified the oil condition into three categories: *pure*, *moderately impure*, and *highly impure*, based on the correlation between pH, turbidity, and color data. The ThingSpeak dashboard provided real-time graphical visualization of these parameters, enabling clear interpretation of oil quality trends.

5.2 SCREENSHOT OF RESULT

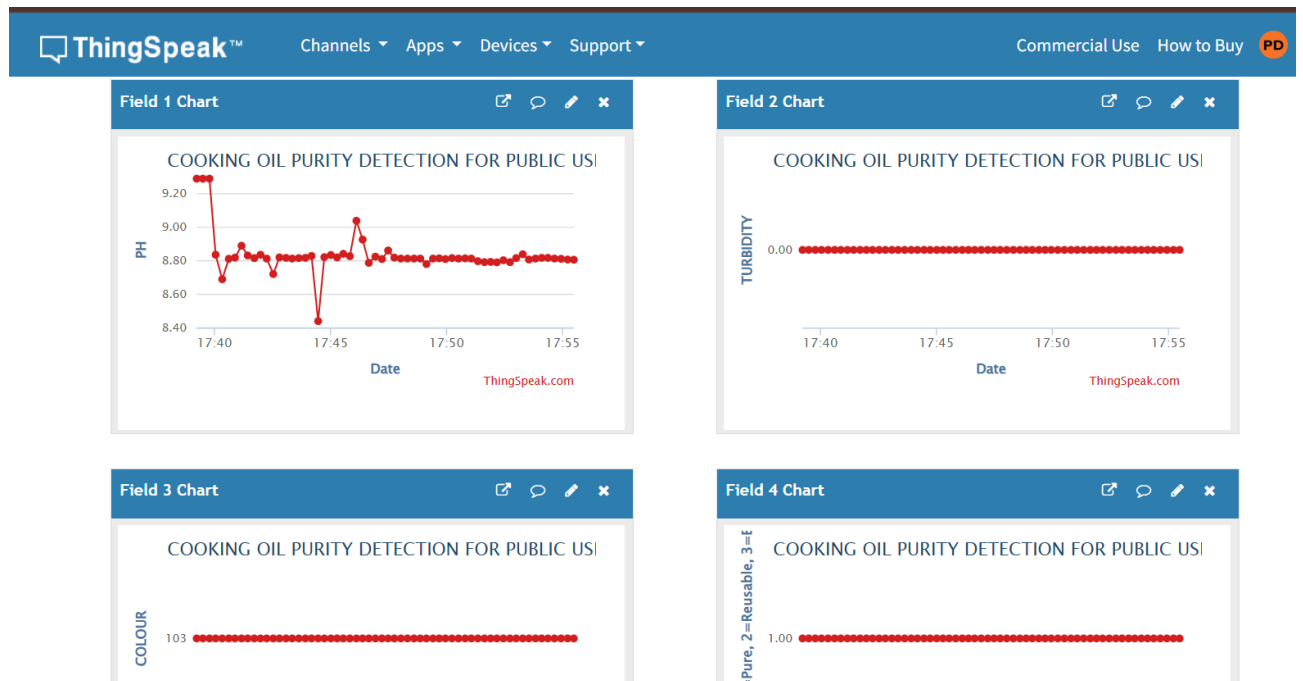


Fig 5.1 Thingspeak Sensors result

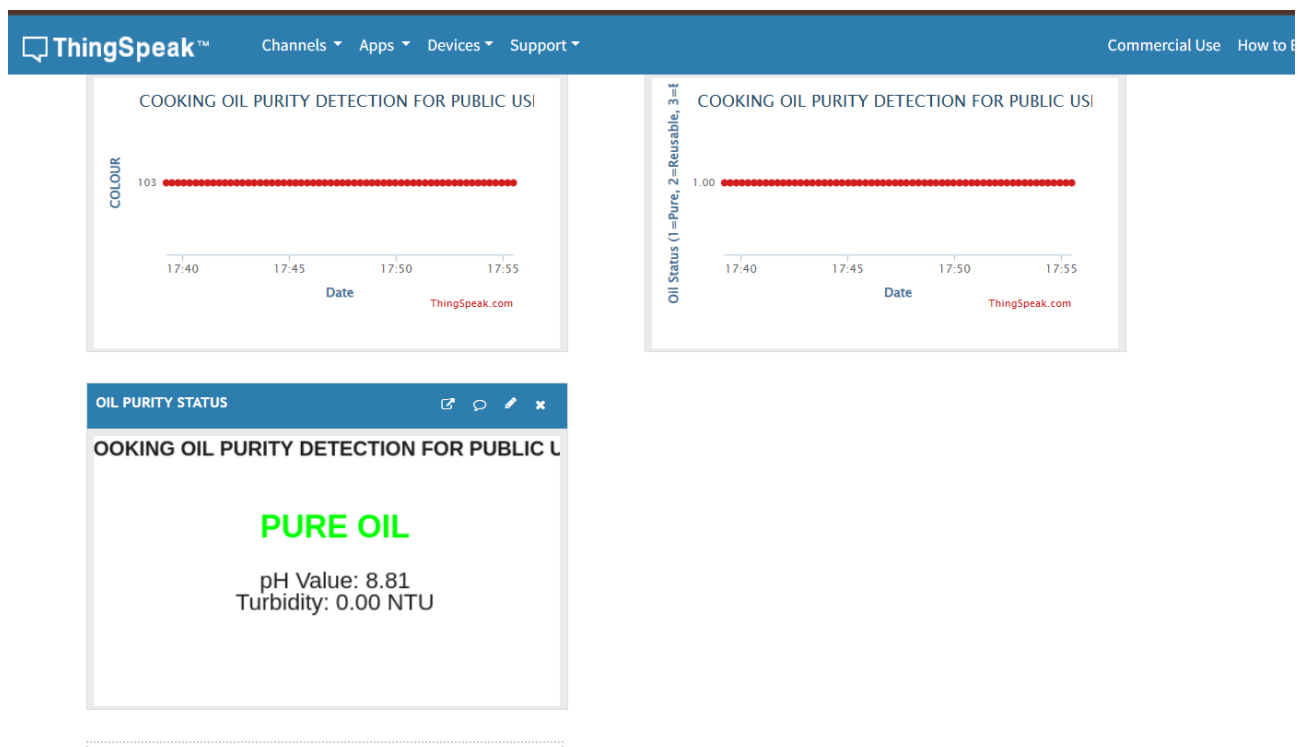


Fig 5.2 Thingspeak Result

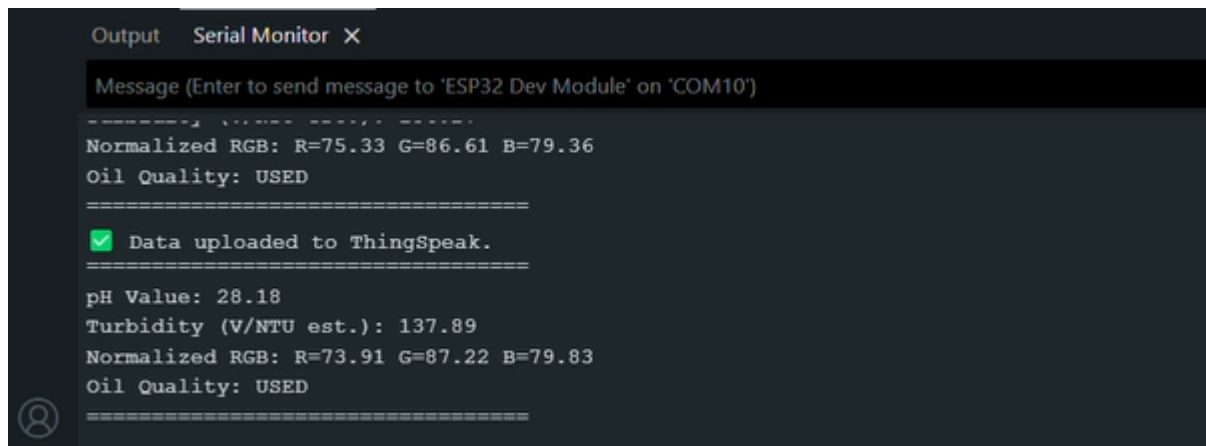


Fig 5.3 Serial Monitor Output

5.3 Summary

The developed system effectively monitored and analyzed cooking oil quality using pH, turbidity, and color sensors.

Experimental testing showed that fresh oil had higher pH, lower turbidity, and stable color readings, while reused oil showed the opposite trends.

These results validated the system's ability to detect chemical and physical degradation in oil samples.

The oil was accurately classified into pure, moderately impure, and highly impure categories.

Data uploaded to ThingSpeak allowed real-time visualization and easy interpretation of oil quality trends.

Overall, the system proved reliable for low-cost, real-time oil purity assessment.

Chapter 6: Conclusion and Future Work

6.1 Summary

This project successfully demonstrated the design and implementation of an **IoT-based oil purity detection system** using an **ESP32 microcontroller** and three key sensors—**pH**, **turbidity**, and **color**. The developed prototype effectively measured oil quality in real time and transmitted the data to the **ThingSpeak cloud platform** for continuous monitoring and visualization.

By applying calibrated threshold values, the system accurately classified oil samples into **pure**, **moderately impure**, and **highly impure** categories. The integration of hardware and software was achieved efficiently, resulting in a compact, low-cost, and reliable solution suitable for **kitchens**, **small-scale food industries**, and **laboratory testing**. The overall system performed consistently during testing, confirming its potential for real-world applications in **oil quality assessment and food safety monitoring**.

6.2 Future Work

While the system achieved its intended objectives, there is scope for further enhancement:

1. **Expanded Parameter Analysis:**

Incorporating additional sensors such as temperature, viscosity, or dielectric constant sensors could provide deeper insights into oil degradation mechanisms.

2. **Automated Alert System:**

Integration of GSM or mobile app notifications could alert users instantly when oil quality falls below safe limits.

3. **Improved Calibration and Accuracy:**

Using higher-resolution ADCs or advanced filtering algorithms may further improve measurement accuracy and reduce noise in sensor readings.

4. **Portable and Energy-Efficient Design:**

Implementing battery optimization techniques or solar charging modules would make the system more suitable for long-term outdoor or industrial deployment.

5. **AI-Based Data Interpretation:**

Machine learning models could be trained using historical data to predict oil degradation trends and recommend optimal replacement intervals.

6.3 Concluding Remarks

In conclusion, the developed **IoT-enabled oil quality monitoring system** successfully met its design objectives, offering an efficient and affordable approach for assessing oil purity. The project highlights how sensor-based automation and cloud connectivity can be leveraged to support **real-time decision-making** and promote **sustainable and safe oil usage**.

- [1] Rizal, R. F., Izzulhaq, M. F., Ramadhan, A. Y., & Permana, D. (2022). Telemonitoring based waste cooking oil quality detection and monitoring system. *Jurnal Akselerasi Telekomunikasi (JARTEL)*, 4(2), 79–85.
<https://jartel.polinema.ac.id/index.php/jartel/article/download/516/223/2355>
- [2] Obaid, R., & Dissanayake, D. (2023). Feasibility of using a cheap colour sensor to detect blends of vegetable oils in avocado oil. *Sensors*, 23(8), 3542.
<https://pmc.ncbi.nlm.nih.gov/articles/PMC10888341/>
- [3] United Nations Environment Programme. (2020). Recycled cooking oils: Assessing risks to human health and the environment. Policy Commons.
<https://policycommons.net/artifacts/1340546/recycled-cooking-oils/1951180/>
- [4] Pradeep, J., Venugopal, V., Gnanamani, J., Dhanasekaran, R., & Mahalakshmy, T. (2025). Pattern of edible oil consumption and practice of reusing oil among households in urban slums of Chennai. *Journal of Family Medicine and Primary Care*, 14(1), 246–251.
https://journals.lww.com/jfmprc/fulltext/2025/01000/pattern_of_edible_oil_consumption_and_practice_of.48.aspx
- [5] S. Herath, H. K. Weerasooriya, D. Y. L. Ranasinghe et al., “Quantitative Assessment of Adulteration and Reuse of Coconut Oil Using Transmittance Multispectral Imaging,” arXiv preprint, Nov. 2020. [researchgate.net](https://www.researchgate.net/publication/357144444)+6[researchgate.net](https://www.researchgate.net/publication/357144444)+6syekh nurjati .ac.id+6[arxiv.org](https://arxiv.org/abs/2104.06310)
- [6] Venturini, F., Sperti, M., Michelucci, U., & Bortolato, M. (2021). Exploration of Spanish Olive Oil Quality with a Miniaturized Low-Cost Fluorescence Sensor and Machine Learning Techniques. arXiv preprint arXiv:2104.06310. <https://arxiv.org/abs/2104.06310>
- [7] Jiang, X., Cheng, J., Yang, F., & Hu, Z. (2023). Visual colorimetric detection of edible oil freshness for peroxides based on nanocellulose. *Foods*, 12(9), 1896.

<https://www.mdpi.com/2304-8158/12/9/1896>

[8] Shin, D., Song, S., & Kim, Y. (2020). Colorimetric sensing of volatile organic compounds produced from heated cooking oils. *ACS Omega*, 5(9), 4510–4517.

<https://pubs.acs.org/doi/10.1021/acsomega.0c05667>

[9] EatingWell. (2025, April 8). Is it safe to reuse cooking oil? Here's what a food scientist says.

EatingWell. This article discusses how repeated heating of oil can produce toxic and carcinogenic compounds, contributing to inflammation, high blood pressure, and even cancer.

<https://www.eatingwell.com/is-it-safe-to-reuse-cooking-oil-11708816>

[10] The Economic Times HealthWorld. (2022, August 25). Sixty per cent of used cooking oil finds its

way back into food stream, says study. Reports carcinogenic free radicals in reused oil and

highlights health risks like heart disease, atherosclerosis, hypertension, liver disorders, and

Alzheimer's. [Referenced via Economic Times] health.economictimes.indiatimes.com

[11] Soundararajan, P., Parthasarathy, S., Sakthivelu, M., Karuppiyah, K. M., Velusamy, P.,

Gopinath, S. C. B., & Raman, P. (2024). Effects of consuming repeatedly heated edible oils on cardiovascular diseases: A narrative review. *Current Medicinal Chemistry*, 31(40),

6630–6648. This review documents that repeated heating leads to trans fats and lipid

peroxidation, increasing cardiovascular disease risk.

<https://pubmed.ncbi.nlm.nih.gov/37877148/>

[12] Hilyana, F. S., Marwoto, P., & Nugroho, S. E. (2024). Educational tool for determining viscosity

coefficient of cooking oil using Arduino UNO. *Jurnal Pendidikan Fisika dan Fisika Terapan*,

10(1), 23–29. <https://journal.unj.ac.id/unj/index.php/jpppf/article/view/40938>

23

APPENDICES

APPENDIX 1: SOURCE CODE

```
#include <Wire.h>
#include <Adafruit_TCS34725.h>
#include <WiFi.h>
#include <ThingSpeak.h>

// ----- WiFi and ThingSpeak -----
const char* ssid = "Galaxy A16 5G 2927";
const char* password = "ttpod123";
WiFiClient client;

unsigned long myChannelNumber = 3070629;
const char* myWriteAPIKey = "6B4U8UT18SXPTRL4";

// ----- Sensor Pins -----
const int pHpin = 34;    // ADC1 on ESP32
const int turbidityPin = 35; // ADC1 on ESP32
const int greenLED = 25;
const int yellowLED = 26;
const int redLED = 27;

// ----- Calibration Constants -----
//  Calibrated pH constants (distilled water ≈ 7)
float pHslope = -1.76;
float pHintercept = 9.29;

//  Turbidity calibration (distilled water ≈ 0 NTU)
float turbiditySlope = -648.88;
float turbidityIntercept = 610.95; // recalibrated intercept for 0 NTU at ~0.94 V

// ----- RGB Sensor -----
Adafruit_TCS34725 tcs = Adafruit_TCS34725(
  TCS34725_INTEGRATIONTIME_154MS,
  TCS34725_GAIN_4X
);
```

```

// ----- Function Prototypes -----
float readPH();
float readTurbidity();
void readRGB(float &rNorm, float &gNorm, float &bNorm);
String classifyOil(float turbidityValue, float colorValue, int &oilStatusNo);
int classifyCode(String quality);

// ----- Setup -----
void setup() {
  Serial.begin(115200);
  delay(2000);
  Serial.println("\n🔧 Cooking Oil Purity Detection System");

  // LED setup
  pinMode(greenLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
  pinMode(redLED, OUTPUT);

  // Initialize TCS34725 color sensor
  if (tcs.begin()) {
    Serial.println("✅ RGB Sensor initialized.");
  } else {
    Serial.println("⚠️ RGB Sensor not found. Check wiring!");
  }

  // WiFi connection
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\n✅ WiFi connected!");
  Serial.print("📶 IP Address: ");
  Serial.println(WiFi.localIP());

```

```

ThingSpeak.begin(client);
}

// ----- Loop -----
void loop() {
  float pH = readPH();
  float turbidity = readTurbidity();
  float rNorm = 0, gNorm = 0, bNorm = 0;
  readRGB(rNorm, gNorm, bNorm);
  float colorValue = (rNorm + gNorm + bNorm) / 3.0;

  int oilStatusNo = 0;
  String oilStatus = classifyOil(turbidity, colorValue, oilStatusNo);

  // Print readings
  Serial.println("=====");
  Serial.printf("pH Value: %.2f\n", pH);
  Serial.printf("Turbidity: %.2f NTU (approx)\n", turbidity);
  Serial.printf("Color Avg: %.2f\n", colorValue);
  Serial.println("Oil Status: " + oilStatus);
  Serial.println("=====");

  // Upload to ThingSpeak
  ThingSpeak.setField(1, (float)pH);
  ThingSpeak.setField(2, (float)turbidity);
  ThingSpeak.setField(3, (float)colorValue);
  ThingSpeak.setField(4, oilStatusNo);

  int x = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);
  if (x == 200) {
    Serial.println("✅ Data uploaded to ThingSpeak.");
  } else {
    Serial.println("⚠ Upload failed, HTTP error: " + String(x));
  }
}

```

```

    delay(15000); // ThingSpeak minimum update interval = 15s
}

```

```

// ----- Functions -----

```

```

// Read pH sensor and convert to pH scale

```

```

float readPH() {
    int sensorValue = analogRead(pHpin);
    float voltage = (sensorValue / 4095.0) * 3.3;
    float pH = pH_Slope * voltage + pH_Intercept;
    return pH;
}

```

```

// Read turbidity sensor

```

```

float readTurbidity() {
    int sensorValue = analogRead(turbidityPin);
    float voltage = (sensorValue / 4095.0) * 3.3;
    float turbidity = turbidity_Slope * voltage + turbidity_Intercept;
    if (turbidity < 0) turbidity = 0;
    return turbidity;
}

```

```

// Read and normalize RGB color sensor data (scaled 0–100)

```

```

void readRGB(float &rNorm, float &gNorm, float &bNorm) {
    uint16_t r, g, b, c;
    tcs.getRawData(&r, &g, &b, &c);
    if (c == 0) c = 1;
    rNorm = ((float)r / c) * 100.0;
    gNorm = ((float)g / c) * 100.0;
    bNorm = ((float)b / c) * 100.0;
}

```

```

// Classification logic + LED control

```

```

String classifyOil(float turbidityValue, float colorValue, int &oilStatusNo) {    27
    String oilStatus;

```

```

if (turbidityValue < 200 && colorValue > 30) {
    oilStatus = "Good Oil";
    digitalWrite(greenLED, HIGH);
    digitalWrite(yellowLED, LOW);
    digitalWrite(redLED, LOW);
    oilStatusNo = 1;
}
else if (turbidityValue > 500 && colorValue > 20) {
    oilStatus = "Used Oil";
    digitalWrite(greenLED, LOW);
    digitalWrite(yellowLED, HIGH);
    digitalWrite(redLED, LOW);
    oilStatusNo = 2;
}
else {
    oilStatus = "Bad Oil";
    digitalWrite(greenLED, LOW);
    digitalWrite(yellowLED, LOW);
    digitalWrite(redLED, HIGH);
    oilStatusNo = 3;
}

return oilStatus;
}

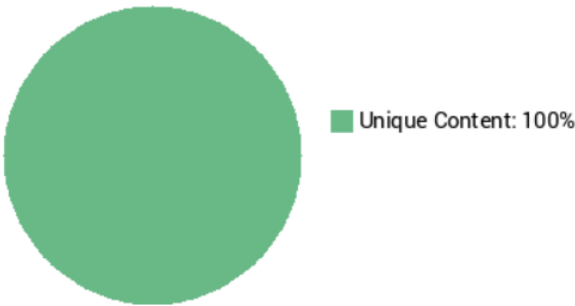
```

```

// Convert text label to numeric code
int classifyCode(String quality) {
    if (quality == "Good Oil") return 1;
    if (quality == "Used Oil") return 2;
    if (quality == "Bad Oil") return 3;
    return 0;
}

```

PLAGIARISM CONTENT



PLAGIARISM SCAN REPORT

Date	November 05, 2025
Exclude URL:	NO
Unique Content	100%
Plagiarized Content	0%
Paraphrased Plagiarism	0